

EXPERIMENT NO. 12

OSSP - Operating Systems and Systems Programming

Course Code: 25CS2104E

Task 1

Producer-Consumer Problem Using Counting Semaphores and POSIX Threads

```
/*
 * Experiment 12 - Task 1: Producer-Consumer Problem using POSIX Threads & Semaphores
 *
 * POSIX Primitives Demonstrated:
 * - pthread_create(), pthread_join()
 * - sem_init(), sem_wait(), sem_post(), sem_destroy()
 * - pthread_mutex_lock(), pthread_mutex_unlock()
 *
 * Data Structure:
 * - Bounded circular buffer of size BUFFER_SIZE.
 *
 * Synchronization:
 * - 'empty' counting semaphore: tracks available empty slots (initially BUFFER_SIZE).
 * - 'full' counting semaphore: tracks filled items available for consumption (initiall
   y 0).
 * - 'buffer_mutex': guarantees mutual exclusion while modifying the circular buffer.
 */

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#ifdef __APPLE__
/*
 * On macOS Darwin, unnamed semaphores (sem_init) are not implemented by the OS (returns
   ENOSYS).
 * This standard counting semaphore wrapper provides the exact POSIX sem_init / sem_wait
   /
 * sem_post / sem_destroy API so the code runs seamlessly across both macOS and Linux.
 */
typedef struct {
    pthread_mutex_t lock;
    pthread_cond_t cond;
    int value;
} sem_t;

static inline int sem_init(sem_t *sem, int pshared, unsigned int value) {
    (void)pshared;
    sem->value = value;
    pthread_mutex_init(&sem->lock, NULL);
    pthread_cond_init(&sem->cond, NULL);
    return 0;
}

static inline int sem_wait(sem_t *sem) {
    pthread_mutex_lock(&sem->lock);
    while (sem->value <= 0) {
        pthread_cond_wait(&sem->cond, &sem->lock);
    }
    sem->value--;
    pthread_mutex_unlock(&sem->lock);
    return 0;
}

static inline int sem_post(sem_t *sem) {
    pthread_mutex_lock(&sem->lock);
    sem->value++;
    pthread_cond_signal(&sem->cond);
    pthread_mutex_unlock(&sem->lock);
    return 0;
}

static inline int sem_destroy(sem_t *sem) {
    pthread_mutex_destroy(&sem->lock);
    pthread_cond_destroy(&sem->cond);
    return 0;
}
```

```
}
#else
#include <semaphore.h>
#endif

#define BUFFER_SIZE 5
#define TOTAL_ITEMS 10

// Circular buffer and index pointers
int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;
int item_count = 0;

// Synchronization primitives
sem_t empty_slots;
sem_t full_slots;
pthread_mutex_t buffer_mutex;

void *producer_thread(void *arg) {
    (void)arg;
    for (int i = 1; i <= TOTAL_ITEMS; i++) {
        int item = i * 100; // item value

        // Wait for an empty slot
        sem_wait(&empty_slots);

        // Enter critical section
        pthread_mutex_lock(&buffer_mutex);

        buffer[in] = item;
        item_count++;
        printf("[Producer] Produced item %3d at buffer[%d] | Total items in buffer: %d\n",
            item, in, item_count);
        in = (in + 1) % BUFFER_SIZE;

        // Leave critical section
        pthread_mutex_unlock(&buffer_mutex);

        // Signal that a new filled slot is available
        sem_post(&full_slots);

        // Small sleep to simulate production time
        usleep(50000); // 50ms
    }
    pthread_exit(NULL);
}

void *consumer_thread(void *arg) {
    (void)arg;
    for (int i = 1; i <= TOTAL_ITEMS; i++) {
        // Wait for a filled slot
        sem_wait(&full_slots);

        // Enter critical section
        pthread_mutex_lock(&buffer_mutex);

        int item = buffer[out];
        item_count--;
        printf(" [Consumer] Consumed item %3d from buffer[%d] | Total items in buffer: %d\n",
            item, out, item_count);
        out = (out + 1) % BUFFER_SIZE;

        // Leave critical section
        pthread_mutex_unlock(&buffer_mutex);

        // Signal that an empty slot is now available
        sem_post(&empty_slots);

        // Small sleep to simulate consumption time
        usleep(80000); // 80ms
    }
    pthread_exit(NULL);
}
```

```

}

int main() {
    pthread_t prod, cons;

    printf("=== Producer-Consumer Problem (Bounded Buffer = %d) ===\n", BUFFER_SIZE);
    printf("Total items to produce and consume: %d\n\n", TOTAL_ITEMS);

    // Initialize synchronization primitives
    pthread_mutex_init(&buffer_mutex, NULL);
    sem_init(&empty_slots, 0, BUFFER_SIZE); // initially BUFFER_SIZE empty slots
    sem_init(&full_slots, 0, 0);           // initially 0 filled slots

    // Create producer and consumer threads
    pthread_create(&prod, NULL, producer_thread, NULL);
    pthread_create(&cons, NULL, consumer_thread, NULL);

    // Wait for both threads to finish
    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    // Clean up
    pthread_mutex_destroy(&buffer_mutex);
    sem_destroy(&empty_slots);
    sem_destroy(&full_slots);

    printf("\nAll items produced and consumed successfully! No overflow or underflow occurred.\n");
    return 0;
}

```

Output:

```

zsh - producer_consumer

rishivarma@MacBook-Air ~ % ./producer_consumer
=== Producer-Consumer Problem (Bounded Buffer = 5) ===
Total items to produce and consume: 10
[Producer] Produced item 100 at buffer[0] | Total items in buffer: 1
[Consumer] Consumed item 100 from buffer[0] | Total items in buffer: 0
[Producer] Produced item 200 at buffer[1] | Total items in buffer: 1
[Consumer] Consumed item 200 from buffer[1] | Total items in buffer: 0
[Producer] Produced item 300 at buffer[2] | Total items in buffer: 1
[Producer] Produced item 400 at buffer[3] | Total items in buffer: 2
[Consumer] Consumed item 300 from buffer[2] | Total items in buffer: 1
[Producer] Produced item 500 at buffer[4] | Total items in buffer: 2
[Consumer] Consumed item 400 from buffer[3] | Total items in buffer: 1
[Producer] Produced item 600 at buffer[0] | Total items in buffer: 2
[Producer] Produced item 700 at buffer[1] | Total items in buffer: 3
[Consumer] Consumed item 500 from buffer[4] | Total items in buffer: 2
[Producer] Produced item 800 at buffer[2] | Total items in buffer: 3
[Consumer] Consumed item 600 from buffer[0] | Total items in buffer: 2
[Producer] Produced item 900 at buffer[3] | Total items in buffer: 3
[Producer] Produced item 1000 at buffer[4] | Total items in buffer: 4
[Consumer] Consumed item 700 from buffer[1] | Total items in buffer: 3
[Consumer] Consumed item 800 from buffer[2] | Total items in buffer: 2
[Consumer] Consumed item 900 from buffer[3] | Total items in buffer: 1
[Consumer] Consumed item 1000 from buffer[4] | Total items in buffer: 0
All items produced and consumed successfully! No overflow or underflow occurred.
rishivarma@MacBook-Air ~ %

```

Task 2

Evaluate Different Bounded Buffer Sizes (Sizes 2, 5, 10)

```

/*
 * Experiment 12 - Task 2: Producer-Consumer Buffer Size Analysis
 *
 * Analyzes the performance, waiting frequency, and throughput across different buffer sizes:

```

```
* - Buffer Size = 2 (Tightly constrained buffer)
* - Buffer Size = 5 (Moderate buffer)
* - Buffer Size = 10 (Large buffer)
*
* Metrics Measured:
* - Producer Blocked Count: times producer had to block waiting for an empty slot.
* - Consumer Blocked Count: times consumer had to block waiting for a filled item.
* - Throughput: items processed per unit time.
*/

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <sys/time.h>

#ifdef __APPLE__
typedef struct {
    pthread_mutex_t lock;
    pthread_cond_t cond;
    int value;
} sem_t;

static inline int sem_init(sem_t *sem, int pshared, unsigned int value) {
    (void)pshared;
    sem->value = value;
    pthread_mutex_init(&sem->lock, NULL);
    pthread_cond_init(&sem->cond, NULL);
    return 0;
}

static inline int sem_wait(sem_t *sem) {
    pthread_mutex_lock(&sem->lock);
    while (sem->value <= 0) {
        pthread_cond_wait(&sem->cond, &sem->lock);
    }
    sem->value--;
    pthread_mutex_unlock(&sem->lock);
    return 0;
}

static inline int sem_trywait(sem_t *sem) {
    pthread_mutex_lock(&sem->lock);
    if (sem->value <= 0) {
        pthread_mutex_unlock(&sem->lock);
        return -1;
    }
    sem->value--;
    pthread_mutex_unlock(&sem->lock);
    return 0;
}

static inline int sem_post(sem_t *sem) {
    pthread_mutex_lock(&sem->lock);
    sem->value++;
    pthread_cond_signal(&sem->cond);
    pthread_mutex_unlock(&sem->lock);
    return 0;
}

static inline int sem_destroy(sem_t *sem) {
    pthread_mutex_destroy(&sem->lock);
    pthread_cond_destroy(&sem->cond);
    return 0;
}
#else
#include <semaphore.h>
#endif

#define TOTAL_ITEMS 15

int g_buffer_size = 5;
int *g_buffer = NULL;
int g_in = 0;
int g_out = 0;
```

```
int g_item_count = 0;

sem_t sem_empty;
sem_t sem_full;
pthread_mutex_t lock_mutex;

int prod_wait_count = 0;
int cons_wait_count = 0;

void *producer(void *arg) {
    (void)arg;
    for (int i = 1; i <= TOTAL_ITEMS; i++) {
        // Check if would block (for statistics)
        if (sem_trywait(&sem_empty) != 0) {
            prod_wait_count++;
            sem_wait(&sem_empty);
        }

        pthread_mutex_lock(&lock_mutex);
        g_buffer[g_in] = i;
        g_in = (g_in + 1) % g_buffer_size;
        g_item_count++;
        pthread_mutex_unlock(&lock_mutex);

        sem_post(&sem_full);
        usleep(15000); // 15ms producer speed
    }
    pthread_exit(NULL);
}

void *consumer(void *arg) {
    (void)arg;
    for (int i = 1; i <= TOTAL_ITEMS; i++) {
        // Check if would block (for statistics)
        if (sem_trywait(&sem_full) != 0) {
            cons_wait_count++;
            sem_wait(&sem_full);
        }

        pthread_mutex_lock(&lock_mutex);
        int item = g_buffer[g_out];
        (void)item;
        g_out = (g_out + 1) % g_buffer_size;
        g_item_count--;
        pthread_mutex_unlock(&lock_mutex);

        sem_post(&sem_empty);
        usleep(25000); // 25ms consumer speed (slower than producer)
    }
    pthread_exit(NULL);
}

void run_test(int buf_size) {
    g_buffer_size = buf_size;
    g_buffer = (int *)malloc(sizeof(int) * g_buffer_size);
    g_in = 0;
    g_out = 0;
    g_item_count = 0;
    prod_wait_count = 0;
    cons_wait_count = 0;

    pthread_mutex_init(&lock_mutex, NULL);
    sem_init(&sem_empty, 0, g_buffer_size);
    sem_init(&sem_full, 0, 0);

    struct timeval start, end;
    gettimeofday(&start, NULL);

    pthread_t p_tid, c_tid;
    pthread_create(&p_tid, NULL, producer, NULL);
    pthread_create(&c_tid, NULL, consumer, NULL);

    pthread_join(p_tid, NULL);
    pthread_join(c_tid, NULL);
}
```

```
gettimeofday(&end, NULL);
double elapsed_ms = (end.tv_sec - start.tv_sec) * 1000.0 + (end.tv_usec - start.tv_usec) / 1000.0;

printf("Buffer Size: %2d | Producer Blocks: %2d | Consumer Blocks: %2d | Elapsed Time: %.2f ms\n",
       buf_size, prod_wait_count, cons_wait_count, elapsed_ms);

free(g_buffer);
pthread_mutex_destroy(&lock_mutex);
sem_destroy(&sem_empty);
sem_destroy(&sem_full);
}

int main(int argc, char *argv[]) {
    printf("=====\n");
    printf("Experiment 12 - Task 2: Comparative Analysis Across Buffer Sizes\n");
    printf("Total Items Processed per Run: %d\n", TOTAL_ITEMS);
    printf("=====\n\n");

    if (argc >= 2) {
        int custom_size = atoi(argv[1]);
        if (custom_size <= 0) custom_size = 5;
        printf("--- Running Single Test with Buffer Size = %d ---\n", custom_size);
        run_test(custom_size);
    } else {
        printf("--- Running Comparative Test for Buffer Sizes: 2, 5, 10 ---\n\n");
        run_test(2);
        run_test(5);
        run_test(10);
    }

    printf("\n=== Analysis and Conclusion ===\n");
    printf("1. Buffer Size = 2:\n");
    printf("   - Smallest capacity creates frequent bottlenecks.\n");
    printf("   - Producer fills the 2 slots rapidly and is frequently blocked waiting.\n");
    printf("2. Buffer Size = 5:\n");
    printf("   - Absorbs moderate production/consumption rate bursts with fewer stalls.\n");
    printf("3. Buffer Size = 10:\n");
    printf("   - Significantly minimizes producer wait times, providing smoother buffering\n");
    printf("   and higher decoupling between producer and consumer.\n");
    printf("=====\n");

    return 0;
}
```

Output:

```

zsh - buffer_size_analysis

rishivarma@MacBook-Air ~ % ./buffer_size_analysis
=====
Experiment 12 - Task 2: Comparative Analysis Across Buffer Sizes
Total Items Processed per Run: 15
=====
--- Running Comparative Test for Buffer Sizes: 2, 5, 10 ---
Buffer Size: 2 | Producer Blocks: 9 | Consumer Blocks: 0 | Elapsed Time: 430.53 ms
Buffer Size: 5 | Producer Blocks: 1 | Consumer Blocks: 0 | Elapsed Time: 430.10 ms
Buffer Size: 10 | Producer Blocks: 0 | Consumer Blocks: 0 | Elapsed Time: 441.23 ms
=== Analysis and Conclusion ===
1. Buffer Size = 2:
- Smallest capacity creates frequent bottlenecks.
- Producer fills the 2 slots rapidly and is frequently blocked waiting.
2. Buffer Size = 5:
- Absorbs moderate production/consumption rate bursts with fewer stalls.
3. Buffer Size = 10:
- Significantly minimizes producer wait times, providing smoother buffering
and higher decoupling between producer and consumer.
=====
rishivarma@MacBook-Air ~ %

```

Task 3**Deadlock Demonstration (Two Threads & Two Mutexes)**

```

/*
 * Experiment 12 - Task 3: Deliberate Deadlock Demonstration
 *
 * Demonstrates the 4 Necessary Coffman Conditions for Deadlock:
 * 1. Mutual Exclusion: Resources cannot be shared simultaneously.
 * 2. Hold and Wait: A thread holding a resource waits to acquire another.
 * 3. No Preemption: Resources cannot be forcibly taken from a thread.
 * 4. Circular Wait: A closed chain of threads exists where each thread
 *    waits for a resource held by the next thread.
 *
 * Execution Flow:
 * - Thread 1 locks Resource 1, then attempts to lock Resource 2.
 * - Thread 2 locks Resource 2, then attempts to lock Resource 1.
 *
 * Safe Timeout Mechanism:
 * - A built-in watchdog timer (alarm) detects the hang after 2 seconds,
 *   prints the diagnostic deadlock state, and exits safely without freezing
 *   the terminal or test runner.
 */

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>
#include <signal.h>

pthread_mutex_t resource1_mutex;
pthread_mutex_t resource2_mutex;

int thread1_holding_r1 = 0;
int thread2_holding_r2 = 0;

void deadlock_watchdog_handler(int sig) {
    (void)sig;
    printf("\n===== \n");
    printf("[WATCHDOG ALERT] SAFE TIMEOUT TRIGGERED (2.0s)\n");
    printf("DEADLOCK CONFIRMED: Execution is completely frozen!\n");
    printf("State Breakdown:\n");
    printf(" * Thread 1: HOLDS Resource 1, BLOCKED WAITING for Resource 2\n");
    printf(" * Thread 2: HOLDS Resource 2, BLOCKED WAITING for Resource 1\n");
    printf(" * Circular Chain: Thread 1 -> Resource 2 -> Thread 2 -> Resource 1 -> Thre\n");
    printf("ad 1\n");
}

```

```
    printf("Neither thread can make progress without external intervention.\n");
    printf("Exiting safely after successful demonstration.\n");
    printf("=====\n");
    exit(0);
}

void *thread1_func(void *arg) {
    (void)arg;
    printf("[Thread 1] Requesting Resource 1...\n");
    pthread_mutex_lock(&resource1_mutex);
    thread1_holding_r1 = 1;
    printf("[Thread 1] SUCCESS: Acquired Resource 1!\n");

    // Sleep to guarantee Thread 2 acquires Resource 2
    printf("[Thread 1] Holding Resource 1; sleeping for 100ms...\n");
    usleep(100000);

    printf("[Thread 1] Now requesting Resource 2 (held by Thread 2)...\n");
    pthread_mutex_lock(&resource2_mutex); // Will block forever here!

    printf("[Thread 1] Unreachable: Acquired Resource 2.\n");
    pthread_mutex_unlock(&resource2_mutex);
    pthread_mutex_unlock(&resource1_mutex);
    pthread_exit(NULL);
}

void *thread2_func(void *arg) {
    (void)arg;
    printf(" [Thread 2] Requesting Resource 2...\n");
    pthread_mutex_lock(&resource2_mutex);
    thread2_holding_r2 = 1;
    printf(" [Thread 2] SUCCESS: Acquired Resource 2!\n");

    // Sleep to guarantee Thread 1 acquires Resource 1
    printf(" [Thread 2] Holding Resource 2; sleeping for 100ms...\n");
    usleep(100000);

    printf(" [Thread 2] Now requesting Resource 1 (held by Thread 1)...\n");
    pthread_mutex_lock(&resource1_mutex); // Will block forever here!

    printf(" [Thread 2] Unreachable: Acquired Resource 1.\n");
    pthread_mutex_unlock(&resource1_mutex);
    pthread_mutex_unlock(&resource2_mutex);
    pthread_exit(NULL);
}

int main() {
    pthread_t t1, t2;

    printf("=== Deliberate Deadlock Demonstration (Two Mutexes, Two Threads) ===\n");
    printf("Initializing resources...\n\n");

    pthread_mutex_init(&resource1_mutex, NULL);
    pthread_mutex_init(&resource2_mutex, NULL);

    // Set up safe watchdog alarm so the program does not hang the terminal forever
    signal(SIGALRM, deadlock_watchdog_handler);
    alarm(2); // 2 second safe timeout for demonstration

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1_mutex);
    pthread_mutex_destroy(&resource2_mutex);

    return 0;
}
```

Output:

```

zsh - deadlock_demo

rishivarma@MacBook-Air ~ % ./deadlock_demo
=== Deliberate Deadlock Demonstration (Two Mutexes, Two Threads) ===
Initializing resources...
[Thread 1] Requesting Resource 1...
[Thread 1] SUCCESS: Acquired Resource 1!
[Thread 1] Holding Resource 1; sleeping for 100ms...
[Thread 2] Requesting Resource 2...
[Thread 2] SUCCESS: Acquired Resource 2!
[Thread 2] Holding Resource 2; sleeping for 100ms...
[Thread 1] Now requesting Resource 2 (held by Thread 2)...
[Thread 2] Now requesting Resource 1 (held by Thread 1)...
=====
[WATCHDOG ALERT] SAFE TIMEOUT TRIGGERED (2.0s)
DEADLOCK CONFIRMED: Execution is completely frozen!
State Breakdown:
* Thread 1: HOLDS Resource 1, BLOCKED WAITING for Resource 2
* Thread 2: HOLDS Resource 2, BLOCKED WAITING for Resource 1
* Circular Chain: Thread 1 -> Resource 2 -> Thread 2 -> Resource 1 -> Thread 1
Neither thread can make progress without external intervention.
Exiting safely after successful demonstration.
=====
rishivarma@MacBook-Air ~ %

```

Task 4

Deadlock Prevention Using Resource Ordering Strategy

```

/*
 * Experiment 12 - Task 4: Deadlock Prevention Using Resource Ordering
 *
 * Deadlock Prevention Technique:
 * - Breaks Coffman Condition #4 (Circular Wait).
 * - Enforces a Global Strict Resource Ordering / Lock Hierarchy:
 *   Resource 1 (Rank 1) must ALWAYS be acquired BEFORE Resource 2 (Rank 2).
 *
 * Execution Flow:
 * - Thread 1 acquires Resource 1 FIRST, then Resource 2 SECOND.
 * - Thread 2 ALSO acquires Resource 1 FIRST, then Resource 2 SECOND.
 * - Because both threads follow the same locking hierarchy, circular wait is impossibl
   e.
 * - Both threads complete their critical sections cleanly without any deadlock.
 */

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1_mutex;
pthread_mutex_t resource2_mutex;

void *thread1_func(void *arg) {
    (void)arg;
    printf("[Thread 1] Step 1: Acquiring Resource 1...\n");
    pthread_mutex_lock(&resource1_mutex);
    printf("[Thread 1] SUCCESS: Acquired Resource 1!\n");

    printf("[Thread 1] Step 2: Acquiring Resource 2...\n");
    pthread_mutex_lock(&resource2_mutex);
    printf("[Thread 1] SUCCESS: Acquired Resource 2!\n");

    // Critical section with both resources
    printf("[Thread 1] *** Executing critical work with both resources ***\n");
    usleep(50000); // 50ms work

    printf("[Thread 1] Step 3: Releasing Resource 2...\n");

```

```
    pthread_mutex_unlock(&resource2_mutex);

    printf("[Thread 1] Step 4: Releasing Resource 1...\n");
    pthread_mutex_unlock(&resource1_mutex);

    printf("[Thread 1] Completed successfully!\n");
    pthread_exit(NULL);
}

void *thread2_func(void *arg) {
    (void)arg;
    // Strict Resource Ordering: Acquire Resource 1 FIRST!
    printf(" [Thread 2] Step 1: Acquiring Resource 1 (respecting hierarchy)...\n");
    pthread_mutex_lock(&resource1_mutex);
    printf(" [Thread 2] SUCCESS: Acquired Resource 1!\n");

    printf(" [Thread 2] Step 2: Acquiring Resource 2...\n");
    pthread_mutex_lock(&resource2_mutex);
    printf(" [Thread 2] SUCCESS: Acquired Resource 2!\n");

    // Critical section with both resources
    printf(" [Thread 2] *** Executing critical work with both resources ***\n");
    usleep(50000); // 50ms work

    printf(" [Thread 2] Step 3: Releasing Resource 2...\n");
    pthread_mutex_unlock(&resource2_mutex);

    printf(" [Thread 2] Step 4: Releasing Resource 1...\n");
    pthread_mutex_unlock(&resource1_mutex);

    printf(" [Thread 2] Completed successfully!\n");
    pthread_exit(NULL);
}

int main() {
    pthread_t t1, t2;

    printf("=== Deadlock Prevention: Resource Ordering Strategy ===\n");
    printf("Global Hierarchy: Resource 1 -> Resource 2\n\n");

    pthread_mutex_init(&resource1_mutex, NULL);
    pthread_mutex_init(&resource2_mutex, NULL);

    pthread_create(&t1, NULL, thread1_func, NULL);
    pthread_create(&t2, NULL, thread2_func, NULL);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    pthread_mutex_destroy(&resource1_mutex);
    pthread_mutex_destroy(&resource2_mutex);

    printf("\nRESULT: Both threads executed and terminated cleanly with ZERO deadlock!\n");
    return 0;
}
```

Output:

```
zsh - deadlock_prevention

rishivarma@MacBook-Air ~ % ./deadlock_prevention
=== Deadlock Prevention: Resource Ordering Strategy ===
Global Hierarchy: Resource 1 -> Resource 2
[Thread 1] Step 1: Acquiring Resource 1...
[Thread 1] SUCCESS: Acquired Resource 1!
[Thread 1] Step 2: Acquiring Resource 2...
[Thread 1] SUCCESS: Acquired Resource 2!
[Thread 1] *** Executing critical work with both resources ***
[Thread 2] Step 1: Acquiring Resource 1 (respecting hierarchy)...
[Thread 1] Step 3: Releasing Resource 2...
[Thread 1] Step 4: Releasing Resource 1...
[Thread 1] Completed successfully!
[Thread 2] SUCCESS: Acquired Resource 1!
[Thread 2] Step 2: Acquiring Resource 2...
[Thread 2] SUCCESS: Acquired Resource 2!
[Thread 2] *** Executing critical work with both resources ***
[Thread 2] Step 3: Releasing Resource 2...
[Thread 2] Step 4: Releasing Resource 1...
[Thread 2] Completed successfully!
RESULT: Both threads executed and terminated cleanly with ZERO deadlock!
rishivarma@MacBook-Air ~ %
```